Defining the Core ATen Opset

compiler FX

SS-JIA

2 Aug 2023

TL;DR

Folks from across Meta internal PyTorch Core, PyTorch Edge, and PyTorch Compiler teams collaborated to review a list of commonly used ATen operators and discussed whether each should be added to the core ATen operator set 265, or be decomposed by the core ATen decomposition table 250.

Our goal is to define a core operator set for the ATen library that fulfills the following criteria:

1. The core ATen operator set can be used as a reference for which ATen ops should be handled by backends or compilers that consume models exported by PT2.
2. The decompositions implied by the core ATen operator set are useful to the vast majority of use-cases
3. The vast majority of use-cases will not want to decompose operators contained in the core ATen operator set

The purpose of having this core operator set is to help PyTorch communicate a stable set of operators that developers can expect their model to produce, therefore constraining the amount of operators that will have to be implemented in a custom runtime or be handled by a custom compiler backend to a manageable quantity. It also facilitates smoother integration with external ML frameworks, such as MLIR, XLA, and ONNX.

We invite you to review our decisions, and provide any feedback you may have!

Context: Why is a Core operator set needed?

As the PyTorch ecosystem grows, there is an increasing demand to convert PyTorch models into specialized representations that can run performantly and efficiently in specific environments. Specific examples today are TorchInductor and Executorch; both consume the same FX Graph representation of a model, but end up producing distinct programs to execute a model in their own distinct runtimes. As more backends are developed, it becomes critical for PyTorch to define a core operator set. This initiative is also a common request from neighboring ML frameworks, such as MLIR/XLA and ONNX so as to facilitate smoother integration with PyTorch.

There are over 3000 operators registered in the ATen library; this is a huge amount of operators that backend authors will have to worry about. To exacerbate the issue, many of these operators are redundant with each other, such as being a slight variant of another operator (e.g. in-place variants, out variants). However, by defining a core operator set, PyTorch is able to communicate a stable set of operators that developers can expect their model to produce, therefore constraining the amount of operators that will have to be implemented in a custom runtime or be handled by a custom compiler backend to a manageable quantity.

Defining the Core Operator Set

The core ATen operator set can be interpreted as the result of reducing the set of all operators registered to ATen through the process of decomposing operators. “Decomposing” an operator involves expressing it as a combination of other operators; such decompositions are currently defined in decomposition.py 45 . During the export process, a default list of decompositions 250 are used; this is known as the core ATen decomposition table. Thus, the core ATen operator set can be interpreted as a list of operators registered to ATen that are not further decomposed.

In general, we define an “operator set” as the list of operators that will be produced when performing a model export with a specific “decomposition table”. Thus, the core ATen operator set is the list of operators that a model can contain when being exported with the core ATen decomposition table.

@SherlockNoMad had previously begun work defining the core ATen opset; the list of ops he identified as belonging in the core IR can be found here: IRs — PyTorch 2.0 documentation 265 . This list was seeded by operators that appeared in 163 open-source models used as PT2 benchmarks from across torchbench, HuggingFace, and TIMM. At this point, the general criteria for determining whether a particular ATen operator can be “easily” decomposed to other ATen operators.

The results we are presenting now is a continuation of Sherlock’s previous work. We follow the same overall process of manually inspecting ops that appear in a body of surveyed models. However, in this iteration we have taken on additional goals of:

- Definition and Codification of the criteria used to evaluate a particular op to determine if it should be part of ATen’s core operator set
- Develop a democratized process where a diverse set of groups across PyTorch interested in this work (i.e. Inductor, Edge, Compiler) can provide input regarding what should/shouldn’t be included in the core operator set, and the discussion and results are transparent to the broader PyTorch community
- Describe the process for evolving this operator set over time; this involves adding new operators to the core set as well as adapting the existing core operator set to changes in function schemas and new operators that are added to ATen

Our end goal is to develop a stable core ATen operator set that fulfills the following goals:

1. The core ATen operator set can be used as a reference for which ATen ops should be handled by backends or compilers that consume models exported by PT2.
2. The decompositions implied by the core ATen operator set are useful to the vast majority of use-cases
3. The vast majority of use-cases will not want to decompose operators in the core ATen operator set

The core operator set represents all ATen operators that we have made an explicit decision to not be decomposed by the Core ATen decomposition table 250 . There are operators that are not decomposed by the core decomposition table, but are also not a part of the Core ATen operator set; this means that these operators have not yet been evaluated or a decision has not yet been made for these operators.

Note that the intention is not for users to be locked in to using the core ATen decomposition table; backends are free to add or remove decompositions as they wish. The core operator set strives to be a common denominator across different use-cases and contexts, but we encourage backends to further fine-tune the decomposition table, and therefore the resulting operator set, for their specific goals.

Results

Folks from across Meta internal PyTorch Core, PyTorch Edge, and PyTorch Compiler came together to review a list of commonly used ATen operators and discussed whether each should be added to the core ATen operator set, or be decomposed by the core ATen decomposition table.

The list of operators under consideration was obtained by extracting operators used by approximately 10,000 `nn.Module` s tested in pytorch-jit-paritybench 4, which is "A test suite to measure TorchScript parity with PyTorch on many `nn.Module` s crawled from popular GitHub projects." The idea was that by looking at operators which are explicitly used in models, we can target the most high-impact ATen operators.

The results of our decisions are summarized below.

Operators Added to the Core ATen Operator Set

For the operators listed below, [core aten] Add ops to core aten set by angelayi · Pull Request #107766 · pytorch/pytorch · GitHub 20 adds the "core" tag to each in `native_functions.yaml`. Since IRs — PyTorch 2.0 documentation 265 is generated by searching through operators with the "core" tage in `native_functions.yaml`, these operators will be eventually reflected in the web page as well.

Operator	Reason / Comment
aten::adaptive_avg_pool1d	avg_pool ops are to be added to core. The adaptive version should be added as well, being a related operator. Decomposing to avg_pool2d involves calculation of kernel sizes based on the input tensor sizes, which in our view strays into the territory of operator implementation.
aten::_adaptive_avg_pool3d	avg_pool ops are to be added to core. The adaptive version should be added as well, being a related operator. Decomposing to avg_pool2d involves calculation of kernel sizes based on the input tensor sizes, which in our view strays into the territory of operator implementation.
aten::_cdist_forward	Decomposition is difficult/impossible; As additional supporting evidence, inductor lowers this

aten::_embedding_bag	Embedding will be added to core. Embedding bag should also be added by extension; it should not be decomposed since the purpose of this op is to not instantiate intermediate embeddings.
aten::_local_scalar_dense	Required since .item() lowers to this.
aten::_native_batch_norm_legit_no_training	Added due to how common the op is. For performance reasons users may not want to decompose batch_norm op. As additional supporting evidence, batch_norm is also part of StableHLO. Note that other functional variants of batch normalization will be added to the core operator set as well.
aten::_pdist_forward	Decomposition is difficult/impossible; As additional supporting evidence, Inductor lowers this
aten::any	Decomposition is difficult/impossible
aten::any.dim	This operator is a variant of any that only reduces a single dim; any and any.dim cannot be represented by each other. Unfortunately, any.dims (a variant which reduces across an arbitrary number of dimensions) does not exist so that any and any.dim can be decomposed to any.dims
aten::avg_pool1d	avg_pool2d already part of core. There is no generic avg_pool operator so avg_pool1d and avg_pool3d should be added as well.
aten::avg_pool3d	avg_pool2d already part of core. There is no generic avg_pool operator so avg_pool1d and avg_pool3d should be added as well.
aten::bitwise_and.Scalar	The .Tensor operator variant already part of core
aten::bitwise_or.Scalar	The .Tensor operator variant already part of core
aten::bitwise_xor.Scalar	The .Tensor operator variant already part of core
aten::ceil	This op also exists in ONNX
aten::clamp.Tensor	Essentially the tensor variant of clamp, which is already in core
aten::cumsum	This op also exists in ONNX
aten::embedding	This op will be difficult to quantize if it is decomposed.
aten::floor	Similar to ceil.
aten::fmod.Scalar	The .Tensor operator variant already part of core
aten::index_put	Decomposition is difficult/impossible
aten::index.Tensor	Decomposition is difficult/impossible

aten::logical_xor	Other logical_* ops are already part of core
aten::mean	Decomposition is difficult/impossible
aten::mean.dim	This operator is a variant of mean that only reduces a single dim; mean and mean.dim cannot be represented by each other. Unfortunately, mean.dims (a variant which reduces across an arbitrary number of dimensions) does not exist so that mean and mean.dim can be decomposed to any.dims
aten::pixel_shuffle	Decomposition is difficult/impossible
aten::prod	Reduction function similar to sum, which is already a part of core
aten::prod.dim_int	Related to prod, but only reduces a single dim. Unfortunately prod.dims does not exist that can express both prod and prod.dim_int
aten::rand	Cannot be decomposed; additionally, can be used in decompositions for various probability distribution generator operators
aten::randperm	Decomposition is difficult/impossible
aten::reflection_pad_1d	Decomposition is difficult/impossible
aten::reflection_pad_2d	Decomposition is difficult/impossible
aten::reflection_pad_3d	Decomposition is difficult/impossible
aten::remainder.Scalar	The .Tensor operator variant already part of core
aten::roll	Decomposition is difficult/impossible
aten::round	Already part of ONNX.
aten::scatter.src	Decomposition is difficult/impossible; Also a part of ONNX
aten::scatter.value	Scalar variant of scatter.src
aten::select_scatter	Reverse operation of select, which is already part of core (and part of StableHLO)
aten::sort	Also exists in stableHLO; cannot be decomposed easily
aten::split_with_sizes	Split is already a part of ONNX. split decomposes to this.
aten::squeeze.dims	Already a part of ONNX
aten::tan	Although this can be decomposed to sin(x)/cos(x), this op also exists ONNX
aten::unsqueeze	Reverse operation of squeeze; it is also used in many decompositions; this is a part of ONNX as well

aten::var.correction	Decomposition is difficult/impossible

Operators for which decompositions will be added to the Core ATen decomposition table

Below contains operators which we have reviewed but have decided that they should be decomposed by default by the core ATen decomposition table. For some of these operators, a decomposition is already registered in the code-base, but it has not yet been added to the core ATen decomposition table.

Operator	Potential Decomp
aten::_trilinear	(i1.unsqueeze(expand1)*i2.unsqueeze(expand2)*i2.unsqueeze(exp
aten::_unsafe_index.Tensor	index.Tensor
aten::_unsafe_view	view()
aten::all.dim	not(any.dim(x))
aten::arange.start	Decomp Exists
aten::atan2	atan(input / other)
aten::baddbmm	inductor has decomp already; make sure it is added to core list
aten::bernoulli	transformation on rand()
aten::bernoulli_.float	transformation on rand()
aten::clamp_max	clamp(x, max=max)
aten::clamp_min	clamp(x, min=min)
aten::copy	_to_copy()
aten::diagonal	decomp for diagonal exists
aten::div.Tensor_mode	div + trunc or round
aten::elu	min(alpha * exp(x) - 1, 0) + max(x, 0)
aten::empty_like	empty()
aten::expm1	exp(x+1)
aten::exponential_	transformation on rand()
aten::floor_divide	floor(divide(x, y))
aten::floor_divide_.Tensor	floor(divide(x, y))
aten::full_like	full()
aten::glu	Split + sigmoid + add
aten::hann_window.periodic	pow(sin(pi * x) / N - 1), 2)
aten::lift_fresh	Gets decomposed to no-op in Core ATen IR
aten::log10	log(x)/log(10)
aten::log1p	log1p (x) = log(1 + x)
aten::log2	log(x)/log(2)

aten::max	return aten::amax(x), aten::argmax(x)
aten::min	return aten::amin(x), aten::argmin(x)
aten::mish_	x * tanh(softplus(x))
aten::normal.Tensor_float	transformation on rand()
aten::normal.Tensor_Tensor	transformation on rand()
aten::pow.Scalar	full + pow.Tensor_Tensor
aten::rand_like	rand()
aten::randint	rand()
aten::randint.low	rand()
aten::randn_like	rand()
aten::resize	view()
aten::split.Tensor	Decomp exists
aten::squeeze	squeeze.dims
aten::std.correction	sqrt(var(x))
aten::sum	sum.dim_IntList(x, □)
aten::unbind	Decomp exists
aten::uniform_	transformation on rand()
aten::unsafe_split.Tensor	split.Tensor
aten::var	var.correction
aten::var_mean.correction	return mean(x), var(x)

We invite you to review these decisions and let us know if there are any operators that you think are misclassified.

The framework we have been using to decide whether an operator should be part of the Core ATen operator set, or decomposed is described below. Note that these should be interpreted more as a set of “rule of thumb”s that guide decisions rather than dictating them.

- The core operator set can only contain functional operators; therefore, in-place and out variant operators are excluded by default
    - During the export process, in-place operators and out variant operators will be replaced by the functional equivalent due to functionalization
        - e.g. aten::gelu_ will be functionalized into aten::gelu
- Core ATen decompositions should be fairly straightforward; a decomposition should not cross into the territory of being an outright implementation of the operator being decomposed. For example, if a decomposition for an operator introduces many additional ops into the graph or requires several computations to produce the decomposition then we should prefer to add the operator as a core operator

    - Introducing many additional ops into the graph also has performance implications such as increasing memory read/writes during computation and needing to allocate more memory for intermediate tensors; for this reason, we prefer to keep decompositions simple
    - Decompositions is not the appropriate layer for complex implementation logic for specific operators; thus if a decomposition is possible but complex, we should prefer to retain the original operator

- Decomposition must be deterministic; if a model is exported once, the decompositions applied must be valid even if the properties of the input tensor (such as tensor sizes and data type) are varied

    - As a general rule, it is fine for decompositions to use the rank of a tensor, since during the export process the rank of a tensor must be fixed.
    - It is also fine for tensors to use the symbolic shape of the tensor for the decomposition

- Whether an operator is included in other stable operator sets, such as ONNX or StableHLO, is also a factor in our decisions. The goal here is to maximize compatibility with external frameworks.

Following Up

Of course, even after this iteration of development, there are still many ATen operators that we have not yet looked at. There are a long tail of specialized operators registered in the ATen library, and exhaustively inspecting each one will be extremely time consuming and may not be practical. We are hoping that after this iteration, most ops that will show up in models in practice will have been accounted for. Nonetheless, it is imperative that we set up a framework for how to evolve this operator set going forward.

To consider additional operators to be added to the core operator set, this can be quite straightforward:

- Internally, we have a workchat group to coordinate discussion around the core ATen operator set; please let me know if you would like to be added. For internal use cases, discussion can occur in these forums.
- Externally, we can monitor open source issues for specific operators that PyTorch clients want to be considered for adding to the core operator set.

A more complex evolution case is when the function schema of existing ops changes, or when additional ops are registered that enable decompositions that were not possible before.

- When a function schema changes, this may result in additional ATen operators being added. If the schema of a core ATen op is changed, causing a variant of that op to be produced, then the added variant may need to be added as a core ATen operator as well. If the variant can be decomposed, then we should decompose it as part of the core ATen decomp table.
    - We are working of an op schema versioning system for PT2 export, similar to the TorchScript op versioning system, which can be relevant here.

- When an additional op is added that enables certain decompositions, this may have the implication that some existing core ATen operators can now be decomposed. In these cases, the best choice may be to add the new operator as a core operator while retaining the existing core operators for the purposes of stability. The old core operators may then be periodically deprecated with significant PyTorch version updates.
  - One similar case to consider is if someone wishes to modify an existing decomposition rule in the Core ATen decomposition table that produces a different set of operators. Even if the new set of operators that are produced are still all a part of the Core ATen operator set, models which trigger the decomposition will now use an alternative operator set. This may have implications for model deployment, where users may employ selective build to include only the minimal set of operators required to execute the model. Thus, re-exporting the model may produce a model which contains operators that are not included in their build process. In this case, the best choice may be to use a similar approach; that is, to prefer keeping existing decompositions in the core ATen decomposition stable until significant Pytorch version updates.

As for evaluating additional operators, we plan to do this on an as-needed basis going forward. If there are operators that you would like us to consider adding to the core ATen operator set, please let us know!

cc: @SherlockNoMad, @Kimishpatel, @larryliu0820, @angelayi

9

5.1k views 16 likes 7 links 6 users 11 min read

nunoplopes Aug 2023

Thank you for putting this together and for making the process a bit more public!

I've 2 questions:
For some ops, the .Scalar overload was added to the core ops (e.g., the bitwise ops), while the .Tensor overload was already there. What's the reasoning to add them to core? Is it because they are common and thus creating a tensor and calling the .Tensor version would be too much overhead?

Regarding functionalization, is there a guarantee that a functional graph will be always generated?
If there's complicated aliasing (e.g., through multiple views), we'll just ship some preconditions to Dynamo as trace guards?

Thanks!



peterbell10 Aug 2023

I have a few concerns regarding the new proposed Core ATen decompositions:

> `aten::_unsafe_index.Tensor` → `index.Tensor`

`aten::_unsafe_index` was created as a hint to inductor that the indices originate from a decomposition rather than a user and as such it should be trusted. This means we don't need to generate a `tl.device_assert` call checking it's in bounds. Decomposing it to `index.Tensor` would result in worse performance.

> `aten::atan2` → `atan(input / other)`

This is incorrect as it doesn't select the correct branch of the `atan` function, e.g. `atan2(-x, -y) != atan2(x, y)` and `atan2(-x, y) != atan2(x, -y)`.

> `aten::diagonal`

Decomposing views into `as_strided` should be discouraged because there is far more semantic information in the `aten::diagonal` call which inductor uses to generate much more efficient code.

> `aten::div.Tensor_mode`, `aten::floor_divide` → floor(divide(x, y))

This decomposition gives different results from python's floor division. Currently inductor does this decomposition, but I don't think it should be baked in for export.

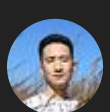
> `aten::expm1`, `aten::log10`, `aten::log1p(x)`, `aten::log2`

These are not just convenience functions, they give more numerical precision so shouldn't be decomposed.

> `aten::var_mean.correction` → `return mean(x), var(x)`

Inductor implements a single pass `var_mean` which already computes the mean, and is currently not CSE'd with `mean`. So this should result in worse performance.

1 Reply 2



SS-JIA Aug 2023

@nunoplopes

> For some ops, the .Scalar overload was added to the core ops (e.g., the bitwise ops), while the .Tensor overload was already there. What's the reasoning to add them to core? Is it because they are common and thus creating a tensor and calling the .Tensor version would be too much overhead?

I believe this is more so because the `.Scalar` variants were initially excluded from the list of ATen operators under consideration when developing the initial list of core operators. As for the reasoning for adding them to core, this was mostly based on the precedence that both the `.Tensor` *and* `.Scalar` variants of some operators were already included in the core IR list (for instance `aten.add.Tensor` and `add.Scalar` are currently both listed on https://pytorch.org/docs/stable/ir.html 4 .

As you mention technically `.Scalar` operator variants can be decomposed by constructing a 1-element tensor from the Scalar argument and then calling the `.Tensor` operator variant. However my personal feeling here is that

1. I'm not sure if constructing/allocating a Tensor during a decomposition is (or should be) allowed
2. Constructing/Allocating a tensor during a decomposition doesn't quite feel right to me

> Regarding functionalization, is there a guarantee that a functional graph will be always generated?
> If there's complicated aliasing (e.g., through multiple views), we'll just ship some preconditions to Dynamo as trace guards?

Unfortunately, I am not an expert on Dynamo or functionalization; so I will redirect this question to @SherlockNoMad .

1



SS-JIA Aug 2023

@peterbell10

>  peterbell10:
>
> `aten::_unsafe_index` was created as a hint to inductor that the indices originate from a decomposition rather than a user and as such it should be trusted. This means we don't need to generate a `tl.device_assert` call checking it's in bounds. Decomposing it to `index.Tensor` would result in worse performance.

Yeah, that's a good point. The main reason we chose to decompose `_unsafe_index` to `index` is because it seems that `_unsafe_index` was introduced specifically for Inductor, and we aren't sure if it would still be applicable to other use cases, such as Executorch.

At least in the case of Inductor, `_unsafe_index` can simply be removed from the decomposition table to preserve `_unsafe_index`.

As for your comments around `atan2`, and `aten::expm1, aten::log10, aten::log1p(x), aten::log2`, those make sense; we will reconsider decomposing those operators by default. Likewise with `aten::diagonal`.

> peterbell10:
>
> > `aten::div.Tensor_mode`, `aten::floor_divide` → floor(divide(x, y))
>
> This decomposition gives different results from python's floor division. Currently inductor does this decomposition, but I don't think it should be baked in for export.

Ah, I see what you mean. For those curious, see here 1. So in that case, it appears that the decomposition should be `a // b = (a - remainder(a, b)) / b` instead.

> peterbell10:
>
> > `aten::var_mean.correction` → `return mean(x), var(x)`
>
> Inductor implements a single pass `var_mean` which already computes the mean, and is currently not CSE'd with `mean`. So this should result in worse performance.

Thus far we have been trying to avoid leaning on "performance" to justify not decomposing an operator whenever possible, since it is often use-case dependent (cc: @Kimishpatel). In this specific case having all of `var_mean`, `var`, and `mean` in the core operator set also seems redundant to me.

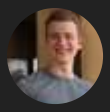
Of course, as with `_unsafe_index` Inductor can avoid including the decomposition of `var_mean` in the export process. But as you mention computed both `var` and `mean` is usually much more efficient in a single pass. So it may be better to not decompose `var_mean` and instead decompose `var -> return var_mean()[1]` and `mean -> return var_mean()[0]`.

1

Lezcano Aug 2023

Why is `aten::mean`'s decomp labelled as "difficult / impossible"? Isn't it actually trivial to do in terms of sum + div.

Also, as Peter has metnioned, it's not clear to me that we want to decompose ops like `log1p`. The initial rationale that went into PrimTorch was "if it's in the C++ STL, it'll be a prim".



bdhirsh Sep 2023

> Regarding functionalization, is there a guarantee that a functional graph will be always generated?
> If there's complicated aliasing (e.g., through multiple views), we'll just ship some preconditions to Dynamo as trace guards?

Hey @nunoplopes - that's right. We will always guarantee a functional graph when you ask for core ATen IR.

More specifically:

- any "internal" mutations in the program will be functionalized away. This includes, say, if you have two tensors that are aliased and you mutate one, we'll make sure the right thing happens.
- "input mutation" support is a bit special, e.g. if you have a model who's forward pass mutates buffers. We will still fully functionalize the graph, but the updated buffers will show up as extra out puts in the forward graph, and export has some metadata telling you specifically which buffers were mutated and what outputs they map to.
- you can imagine some crazier situations, where e.g your model takes in two inputs that are aliased *to each other*, and your model mutates them. This is actually supported in torch.compile, but banned when trying to export an aten graph (because it requires complicated calling convention changes, and is generally not common).

nunoplopes Sep 2023

Thank you!

It's just that we sometimes see some bytecode leftovers when compiling models. I'm thinking what we can do about that. Python bytecode isn't the nicest thing to deal with.
Anyway, I'll come back with some concrete proposals/questions.

1

11 days later



SS-JIA Sep 2023

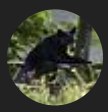
@Lezcano apologies for the late reply, I just returned from a vacation.

> Why is `aten::mean` 's decomp labelled as “difficult / impossible”? Isn't it actually trivial to do in terms of sum + div.

From a mathematical perspective, it's quite simple as you've pointed out. The challenge is that the decomposition needs to determine the number of elements to divide the sum by, which would require capturing that value as a symbolic Int and dividing the sum by it. Generally my aim was to avoid using symbolic ints representing tensor properties in computations; I will update the reasoning to reflect this.

> Also, as Peter has metnioned, it's not clear to me that we want to decompose ops like `log1p` . The initial rationale that went into PrimTorch was “if it's in the C++ STL, it'll be a prim”.

This is a fair point. I'm leaning towards taking this stance as well.



Lezcano Sep 2023

Then you should probably change the paragraph

> It is also fine for tensors to use the symbolic shape of the tensor for the decomposition

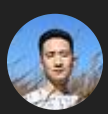
as it reads as if dividing by a symint were allowed.

1



SS-JIA Sep 2023

@Lezcano sounds good, I plan on going through the list and providing more technical descriptions of why specific decompositions are not allowed.



SS-JIA Sep 2023

@peterbell10 and @Lezcano thank you for your feedback. Based on your comments, we are making the following changes to the list of identified core operators.

1. Operators which cleanly map to hardware intrinsics will be promoted to the core operators. The same treatment is applied to operators where decomposing it will impact the numerical precision/stability of the output. In accordance with this, the following operators which were previously decomposed are now added to core:
    - `aten::trunc`
    - `aten::expm1`
    - `aten::log10`
    - `aten::log1p`
    - `aten::log2`
    - `aten::atan2`
2. `div.Tensor_mode` and `div.Scalar_mode` has been added as a "core" operator. The `"trunc"` and `"floor"` rounding modes are more complex to decompose than initially thought, as both need to handle floating point and integer data types separately, and "floor" in particular is quite complex to decompose as it needs to replicate Python's floor division behaviour. The decomposition for thisoperator would be too similar to an outright implementation of the operator, which is why it is preferable to add it as a "core" operator.

Despite these changes, there are still some additional considerations I am working through.

1. For `aten::diagonal`, as @peterbell10 called out decomposing into `as_strided` is not ideal. I am in favor of moving this to a core op as well, but need to confirm this decision internally.
2. We are still undecided on how to handle `var_mean.correction`. We can remove this decomposition for Inductor, but need to determine if there is a need to add the op as core so that the single pass algorithm can be acessed.
3. As a general point for the `.Scalar` variant of ops, should they be decomposed to using `full` to construct tensor argument using the Scalar argument, then call the `.Tensor` variant?

1

Core ATen opset backward & forward compatibility policy 4

11 months later



lingzhi98 1 Aug 2024

Many ops has scalar_tensor version in native_functions.yaml, but are not included in core aten IR. Some of them has same results between scalar_tensor version and scalar version, like bitwise_and:

```
a = 5
b = torch.tensor([1, 2, 3], dtype=torch.int)
# scalar_tensor version, results are [1, 0, 1]
res0 = torch.bitwise_and(a, b)
```

```
# scalar version, results are [1, 0, 1]
res1 = torch.bitwise_and(b, a)
```

Some are different, like remainder and bitwise_shift:

```
a = 5
b = torch.tensor([1, 2, 3], dtype=torch.int)
# scalar_tensor version, results are [0, 1, 2]
res0 = torch.remainder(a, b)
# scalar version, results are [1, 2, 3]
res1 = torch.remainder(b, a)
```

Is it missing or is there some other reason? Can we add all of these op in core aten IR? Or we only add the ops which results are different, try to convert scalar_tensor version to scalar version (maybe we can use decomposition, but it seems not satisfy the definition of decomposition) if results are same?

5.1k views 16 likes 7 links 6 users 11 min read

Reply

✨ Related topics

Topic	Replies	Activity
Why PyTorch does not need a new standardized operator set compiler	3	Jul 2024
Core ATen opset backward & forward compatibility policy	5	Jan 2024
What is the right way to use decompositions in dispatch mode? Tensor subclass	0	19d
Set of ops for a backend to register hardware-backends	10	Mar 2023
[RFC] Adding Triton Backend for Aten operators hardware-backends	0	Nov 2024